

LinkedList - Complete Notes

Simple revision notes written in the way we discussed in chat: why - how - when - interview.

This chapter explains LinkedList from zero to interview level with examples, diagrams, complexity, and common mistakes.

1. Why was LinkedList created?

ArrayList is very good for reading by index, but it becomes slow when you insert or delete in the middle because elements must shift. LinkedList was created for situations where inserting and deleting happen frequently.

Think of a train. If you want to add or remove a coach, you mostly change the links between coaches. You do not need to move every coach one by one. That is the main idea behind LinkedList.

Main idea
ArrayList = better for fast reading
LinkedList = better for frequent insert/delete
LinkedList uses links instead of shifting elements

2. What is LinkedList?

LinkedList is a linear data structure where each element is stored in a node. Each node knows the previous node and the next node. Because of that, Java can move both forward and backward.

Simple node idea:

```
prev <- [ data ] -> next
```

In Java, LinkedList is commonly used as a doubly linked list. That means each node contains three parts: previous reference, data, and next reference.

3. Node structure

One node conceptually looks like this:

```
+-----+-----+-----+
| prev |  data  | next |
+-----+-----+-----+
```

```
prev -> previous node
data -> value
next -> next node
```

Because of the two links, LinkedList can move in both directions. This is why it is called a doubly linked list.

4. Real-life example

Imagine a chain of people standing in line and each person knows the person before them and after them. If one person leaves, the neighbors simply connect to each other. That is a very good mental picture for LinkedList.

Visual idea
10 <-> 20 <-> 30 <-> 40

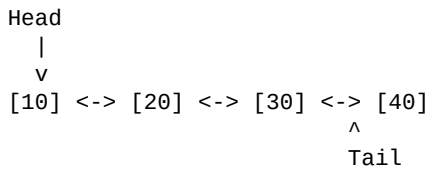
Each node keeps a link to previous and next

No shifting like ArrayList

5. How LinkedList stores data

LinkedList does not keep elements in continuous memory like an array. Each node can be in a different place in memory, but the links connect them. So Java follows the links instead of using an index directly.

Conceptual picture:



The first node is usually called head and the last node is called tail.

6. Why is get(index) slow?

ArrayList can jump directly to an index. LinkedList cannot do that because its nodes are connected one by one. To reach index 500, Java may need to travel through 500 nodes. That is why get(index) is $O(n)$.

Easy way to remember
ArrayList: direct jump
LinkedList: walk node by node

7. Why are insertions and deletions fast?

If the position is already known, LinkedList only changes references. For example, if 20 is removed from 10 <-> 20 <-> 30, Java changes 10 to point to 30 and 30 to point to 10. No shifting of all later elements is needed.

Before:
10 <-> 20 <-> 30

Remove 20:
10 <-> 30

That is the real power of LinkedList: link updates instead of shifting.

8. Head and tail

LinkedList keeps track of the first and last node. That is why addFirst(), addLast(), removeFirst(), and removeLast() are fast.

Head -> [10] <-> [20] <-> [30] <- Tail

When we add or remove at the ends, Java mostly changes the head or tail reference and one or two node links.

9. Common operations

Operation	What happens	Complexity
addFirst()	Insert at beginning	$O(1)$
addLast()	Insert at end	$O(1)$
removeFirst()	Remove beginning node	$O(1)$
removeLast()	Remove last node	$O(1)$
get(index)	Traverse to node	$O(n)$

contains()	Search node by node	O(n)
------------	---------------------	------

10. Why is LinkedList not good for random access?

Because to access a particular index, Java has to start from the head or tail and move step by step. The middle element is not directly reachable by index like an array. So if you often do `list.get(500000)`, LinkedList is not a good choice.

Interview trap
Fast insert/remove does not mean fast search
LinkedList is great only when the use case matches its strengths

11. ArrayList vs LinkedList

ArrayList	LinkedList
Dynamic array	Doubly linked list
Fast <code>get(index)</code>	Slow <code>get(index)</code>
Middle insert/remove slow	Middle insert/remove faster when position known
Good for more reads	Good for more modifications

12. LinkedList as Queue

LinkedList is often used to implement Queue because it supports fast add at the end and fast remove at the beginning. That matches the FIFO behavior of a queue.

```
Queue<Integer> q = new LinkedList<>();
q.offer(10);
q.offer(20);
q.poll(); // removes 10
```

This is one reason LinkedList is important. It can act like a list, queue, or deque depending on how you use it.

13. Time complexity

Operation	Complexity	Why
<code>get(index)</code>	O(n)	Traverse node by node
<code>addFirst()</code>	O(1)	Head update
<code>addLast()</code>	O(1)	Tail update
<code>removeFirst()</code>	O(1)	Head update
<code>removeLast()</code>	O(1)	Tail update
<code>contains()</code>	O(n)	Sequential search

14. Common mistakes

A common mistake is assuming LinkedList is always faster than ArrayList. That is not true. If you need random access by index, ArrayList is usually better. Another mistake is forgetting that LinkedList still needs traversal to reach the insertion position.

So the right answer in interviews is not 'LinkedList is faster' or 'ArrayList is faster'. The right answer is: choose the collection based on your main operation.

15. Interview questions

Why is LinkedList called doubly linked? Why is get(index) slow? Why is addFirst() fast? When would you choose LinkedList over ArrayList? Why is LinkedList useful for Queue?

16. Final revision sheet

Topic	Simple meaning
LinkedList	Nodes connected by links
Node	Stores prev, data, next
Head	First node
Tail	Last node
get(index)	Slow because of traversal
addFirst/addLast	Fast because links change
removeFirst/removeLast	Fast because links change
Best use	Frequent insert/delete

One-line memory: LinkedList = nodes with links. Fast at ends, slow for index-based access.